

Université de Montpellier

Faculté des Sciences

HAI704I · Architectures logicielles distribuées

TP 1 — Java RMI

Chanaud Léo

22205435

MASTER 1 IASD

2026 - 2027

Table des matières

1 Introduction **2**

2 Etape A : Java RMI « vanilla » **3**
2.1 Etape A1 : un premier objet distant, l'animal 3
2.2 Etape A2 : l'espèce, une donnée qui traverse le réseau 4
2.3 Etape A3 : le dossier de suivi, à moi de décider 5
2.4 Etape A4 : le cabinet vétérinaire, un point d'entrée unique 6
2.5 Etape A5 : créer un patient depuis le client 7
2.6 Etape A6 : alertes, le serveur rappelle ses clients 7
2.7 Etape A7 : ce qui traverse vraiment le réseau 8
2.8 Etape A8 : une application utilisable, l'interface en ligne de commande 8

3 Etape B : assembler l'application avec Spring Boot **10**
3.1 Repartir d'un projet Maven propre 10
3.2 Ce que Spring Boot a changé concrètement 11
3.3 Un problème rencontré, le client graphique qui refusait de s'ouvrir 11
3.4 Bilan 12

4 Etape C : une interface graphique pour le client **12**
4.1 Ce qui a été implémenté 12
4.2 Des améliorations ajoutées de ma propre initiative 13
4.3 Validation 13
4.4 Axes d'amélioration envisageables 14

5 Synthèse **14**

6 Usage des outils d'intelligence artificielle générative **15**

7 Références **15**

1 Introduction

L'informatique actuelle est complexe et repose sur de nombreux concepts qui se chevauchent et qui ne sont pas toujours faciles à appréhender. Il est alors courant de concevoir des programmes dont les différentes parties peuvent communiquer entre elles et être réutilisées dans différents contextes. Les architectures distribuées permettent notamment de répartir les traitements et les données entre plusieurs processus, voire plusieurs machines.

Dans le cadre de mon premier TP en **Architecture Logicielle Distribuée** (HAI704I), j'ai eu l'occasion de découvrir le concept de **RMI (Remote Method Invocation)**. Ce mécanisme propre à Java permet à un objet situé dans une JVM d'appeler des méthodes sur un objet situé dans une autre JVM, comme s'il s'agissait d'un objet local. Il permet ainsi de mettre en place une communication entre un client et un serveur tout en conservant une approche orientée objet.

Pour rappel, l'objectif de ce TP était de construire, en partant de rien, une application complète en **Java RMI natif**, représentant un cabinet vétérinaire. Les patients et leurs dossiers vivent dans la JVM du serveur et sont manipulés par des clients à distance, donc dans un processus différent pour vulgariser. Il fallait notamment mettre en place un objet distant, un cabinet servant de point d'entrée unique vers les patients, la création et la recherche de patients ainsi qu'un système d'**alertes par rappel des clients**.

Le TP avait également pour objectif de nous faire réfléchir à la conception d'une application distribuée : distinguer les objets transmis par valeur de ceux transmis par référence, séparer clairement le client de l'implémentation du serveur et gérer les différents cas d'erreur pouvant apparaître lors des communications à distance. L'application devait également rester utilisable grâce à une interface en ligne de commande séparée de la logique métier et de la partie RMI.

Le travail était organisé en plusieurs étapes progressives, permettant de construire l'application et d'aborder les différents concepts de RMI au fur et à mesure. C'est ainsi que sera organisé ce compte-rendu : chaque étape présentera ce qui a été implémenté, mes choix de conception et leur justification, ainsi que les problèmes rencontrés lorsque cela est pertinent. Une attention sera également portée à la structure du projet, à sa reproductibilité et à la possibilité pour un tiers de reconstruire et d'utiliser l'application.

Comme demandé dans le compte-rendu, des schémas seront utilisés pour appuyer mes propos et des extraits de code seront présentés lorsque cela sera utile. La plupart du code étant commenté et l'architecture étant détaillée dans le `README.md`, seuls les passages nécessaires à la compréhension des différents mécanismes seront présentés.

Enfin, une partie importante du travail concernait la communication et la reproductibilité du projet. Le compte-rendu devait permettre d'expliquer les choix réalisés, de présenter l'architecture et les mécanismes importants à l'aide de schémas et d'extraits de code pertinents, mais également de fournir suffisamment d'informations pour qu'une autre personne puisse reconstruire et exécuter l'application à partir du projet et de son README. Une vidéo de démonstration ainsi qu'une déclaration concernant l'utilisation éventuelle d'outils d'intelligence artificielle complétaient également le rendu.

2 Etape A : Java RMI « vanilla »

Avant d'attaquer étape par étape, il faut garder à l'esprit que cette partie représente la quasi-totalité du travail noté : le barème porte uniquement sur elle (20 points), et c'est elle qui construit, brique après brique, toute la logique de l'application. Les parties B (migration vers Spring Boot) et C (interface graphique) viennent seulement rhabiller ce qui est bâti ici, sans jamais le réécrire ; elles seront donc traitées sans réexplications dans les sections suivantes.

Des captures d'écran prises à chaque point de contrôle sont rangées dans `screenshots/` à la racine du dépôt (`A1.1.png`, `A2.1.png`, etc.) ; j'y renvoie au fil du texte plutôt que de décrire une console en mots.

On commence par créer l'architecture en trois projets demandée par le sujet (étape A0) :

```
cabinet-veterinaire/
--> common/veterinaire/    -> contrat partagé par le client et le serveur
--> server/veterinaire/    -> le serveur uniquement
--> client/veterinaire/    -> le client uniquement
```

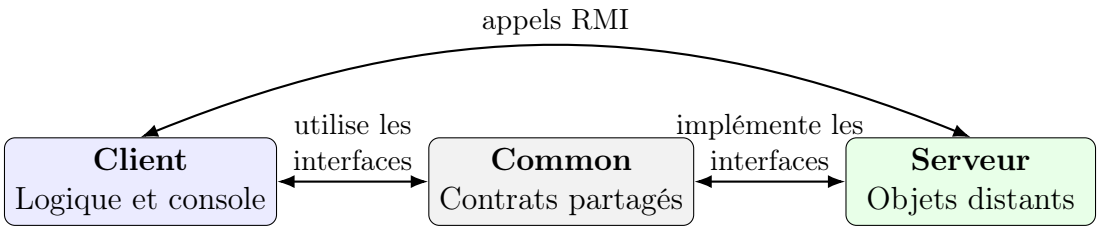


FIGURE 1 – Séparation entre les contrats partagés, le client et le serveur.

2.1 Etape A1 : un premier objet distant, l'animal

Chaque patient du cabinet est un animal, caractérisé au minimum par un nom, le nom de son maître, sa race et son espèce. Dans cette première étape, l'espèce n'est encore qu'une simple chaîne de caractères : on verra à l'étape A2 pourquoi ce choix ne tient pas longtemps.

À chaque fois qu'on veut rendre un objet accessible à distance, on commence par écrire son **contrat distant** : une interface qui étend `Remote` et qui déclare les méthodes qu'un client pourra appeler. Chacune de ces méthodes déclare `RemoteException`, une exception *vérifiée* propre à RMI : elle oblige l'appelant à prévoir le cas où la communication échoue (réseau coupé, serveur arrêté...), ce qu'une exception classique comme `NullPointerException` ne permet pas de distinguer d'une simple erreur de programmation locale.

Listing 1 – Contrat distant minimal de l'animal (`common/veterinaire/IAnimal.java`)

```java
public interface IAnimal extends Remote {
    String getNom() throws RemoteException;
    String getNomMaitre() throws RemoteException;
    String getRace() throws RemoteException;
    String getEspece() throws RemoteException; // simple String pour l'instant
}
```

Le servant `Animal.java`, côté serveur, implémente cette interface et hérite d'`UnicastRemoteObject`, ce qui déclenche son export dès la construction. Le client, lui, ne connaît que l'interface : il récupère le registre, y cherche le nom `"animal"`, transtype le résultat en `IAnimal` et appelle les getters comme s'il s'agissait d'un objet local.

Pour répondre au point de contrôle A, j'affiche `stub.getClass().getName()` et

`Proxy.isProxyClass(stub.getClass())`. On lit effectivement un nom de la forme `jdk.proxy2.$Proxy0`, et le second affichage renvoie `true`. Cela confirme le point du CM 02 : l'objet que détient réellement le client n'est ni une copie de la classe `Animal`, ni cette classe elle-même (le client ne la compile même pas), mais un **proxy dynamique** engendré à l'exécution par le runtime RMI, qui implémente `IAnimal` et relaie chaque appel vers le servant par le réseau. C'est ce qui rend `IAnimal` indispensable : sans interface commune, rien ne permettrait de transtyper ce proxy vers quoi que ce soit d'utilisable.

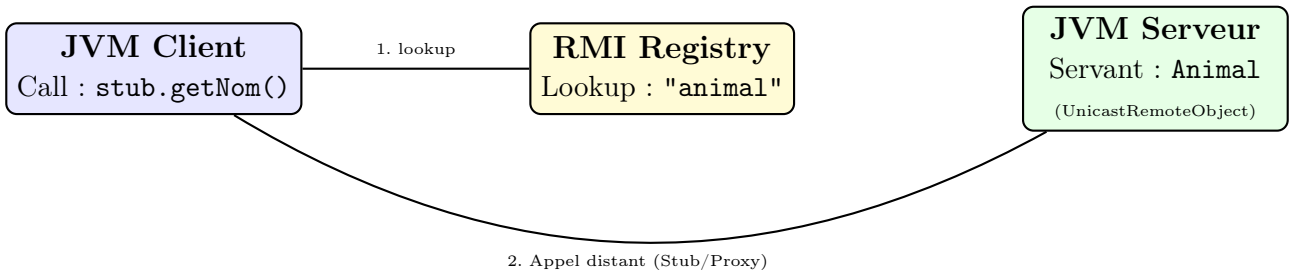


FIGURE 2 – Architecture RMI de base pour l'objet distant `Animal` (point de contrôle A, voir `screenshots/A1.1.png`).

2.2 Etape A2 : l'espèce, une donnée qui traverse le réseau

L'espèce devient une classe à part entière dans `common` (un nom et une espérance de vie), plutôt qu'une simple chaîne. Le choix du sujet est explicite : on veut pouvoir *consulter* l'espèce d'un animal, mais pas la modifier à distance. `Especе` n'étend donc pas `Remote` : elle implémente `Serializable`, et `IAnimal.getEspece()` la retourne directement (sans `RemoteException` supplémentaire sur ses propres méthodes, puisqu'aucune d'elles n'est appelée à distance).

Listing 2 – `common/veterinaire/Especе.java`, avec les setters utilisés pour l'expérience du point de contrôle B

```
public class Especе implements Serializable {
    private String nom;
    private int esperanceDeVie;

    public Especе(String nom, int esperanceDeVie) { ... }
    public String getNom() { return nom; }
    public int getEsperanceDeVie() { return esperanceDeVie; }

    // Setters : ils ne modifient QUE la copie locale (voir explication ci-dessous)
    public void setNom(String nom) { this.nom = nom; }
    public void setDureeDeVie(int esperanceDeVie) { this.esperanceDeVie =
        esperanceDeVie; }
}
```

C'est là que la distinction `Serializable` / `Remote` devient concrète. Un objet `Remote` exporté (comme `Animal`) reste physiquement dans la JVM du serveur : ce que le client reçoit est un *moyen de le joindre* (le proxy vu en A1), donc un seul objet existe, et toute modification faite par un client se voit immédiatement par tous les autres. Un objet `Serializable` (comme `Especе`) est au contraire *recopié* : son état est sérialisé, transformé en flux d'octets, envoyé sur le réseau, puis désérialisé côté client en une **instance toute neuve**, indépendante de l'originale. C'est « envoyer une photo » plutôt que « envoyer un moyen de joindre l'objet » (CM 02, section 5). Les setters ajoutés ci-dessus servent justement à le démontrer :

ils permettent de modifier la copie reçue par le client, en sachant pertinemment que cette modification n'ira jamais au serveur.

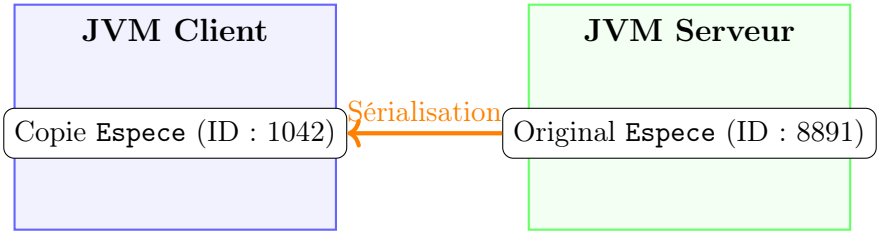


FIGURE 3 – Différence d'identité d'objet (`identityHashCode`) suite à la sérialisation.

Le client de cette étape suit un déroulé que l'on retrouvera à l'identique dans toutes les étapes suivantes :

1. lire l'hôte en argument : `String host = (args.length < 1) ? null : args[0];`
2. se connecter au registre : `Registry registry = LocateRegistry.getRegistry(host);`
3. récupérer le stub : `IAnimal stub = (IAnimal) registry.lookup("animal");`
4. afficher l'animal et son espèce, avec `System.identityHashCode(espece)` ;
5. modifier la copie locale de l'espèce (`setNom`, `setDureeDeVie`) ;
6. redemander l'espèce au serveur (`stub.getEspece()`) et comparer : le nom et la durée de vie redeviennent ceux d'origine, et le second `identityHashCode` diffère du premier.

C'est exactement ce que demande le point de contrôle B, et la preuve est visible sur `screenshots/A2.2.png` et `A2.3.png` : deux identités d'objet différentes, et une valeur serveur jamais affectée par la mutation côté client.

2.3 Etape A3 : le dossier de suivi, à moi de décider

C'est la décision de conception centrale du TP : chaque animal possède désormais un dossier de suivi (un état de santé, et un historique d'observations), et il fallait choisir s'il devait être une classe sérialisable (copié à chaque lecture) ou un objet distant (partagé par tous les clients).

J'ai choisi d'en faire un objet **distant** à part entière : `IDossierSuivi` étend `Remote`, et son servant `DossierSuivi` est exporté *séparément* de l'animal qui le porte (il est créé et exporté dans le constructeur d'`Animal`, mais reste un objet distant indépendant, avec son propre stub). Tous les clients qui appellent `IAnimal.getDossierSuivi()` sur le même animal reçoivent donc un stub pointant vers le **même servant** côté serveur : si un vétérinaire ajoute une observation, elle est immédiatement visible par les autres, sans qu'aucun rafraîchissement explicite ne soit nécessaire.

Listing 3 – `common/veterinaire/IDossierSuivi.java`

```
public interface IDossierSuivi extends Remote {
    String getEtatSante() throws RemoteException;
    void setEtatSante(String etatSante) throws RemoteException;
    List<String> getHistorique() throws RemoteException;
    void ajouterObservation(String observation) throws RemoteException;
}
```

Ce choix part d'une hypothèse que j'assume et qui mérite d'être explicitée : un cabinet vétérinaire regroupe en réalité plusieurs praticiens, chacun connecté depuis son propre poste (donc son propre processus client), et qui doivent voir le même dossier évoluer en temps réel. L'alternative — un `DossierSuivi Serializable`, sur le modèle d'`Especе` — aurait donné à chaque client sa *propre* copie : des modifications

locales, jamais partagées. Un choix tout à fait défendable si le suivi n'avait pas besoin d'être consulté à plusieurs, mais qui ne correspond pas à l'usage visé ici.

Le point de contrôle C se vérifie directement : un client modifie l'état de santé, puis une seconde lecture — soit par un second `lookup`, soit depuis un autre client — montre bien la modification (voir `screenshots/A3.1.png` et `A3.2.png`). C'était la preuve attendue que le dossier est bien un objet partagé, et non une copie.

On arrive ainsi à une première base solide : un animal avec ses caractéristiques et un dossier de suivi partagé. La limite devient vite visible : côté client, chaque animal doit encore être nommé et recherché individuellement dans le registre un par un. Il manque un point d'entrée unique qui connaîtrait toute la collection de patients.

2.4 Etape A4 : le cabinet vétérinaire, un point d'entrée unique

Publier chaque animal séparément dans le registre ne passerait pas à l'échelle : il faudrait en connaître le nom à l'avance. Le `CabinetVeterinaire` résout ce problème : c'est désormais le **seul** objet publié, et il connaît sa collection de patients (une simple `ArrayList<IAnimal>` pour le moment).

Listing 4 – Recherche par nom dans `server/veterinaire/CabinetVeterinaire.java`

```
public IAnimal rechercherParNom(String nom) throws RemoteException {
    for (IAnimal a : patients) {
        if (a.getNom().equals(nom)) return a;
    }
    System.out.println("Ce nom n'existe pas dans la base de donnees du cabinet
        veterinaire.");
    return null;
}
```

La recherche se fait au plus simple : on parcourt la liste et on compare les noms avec `equals`. Une perspective d'amélioration évidente serait d'ignorer la casse, voire de suggérer un patient proche lorsque l'orthographe ne correspond pas exactement (une distance de Levenshtein, par exemple) — mais ce n'était pas l'objet de cette étape, et le comportement reste **défini et documenté** comme l'exige le point de contrôle D : une recherche infructueuse renvoie `null` côté client, et le serveur journalise l'échec de son côté plutôt que de laisser l'appelant deviner ce qui s'est passé.

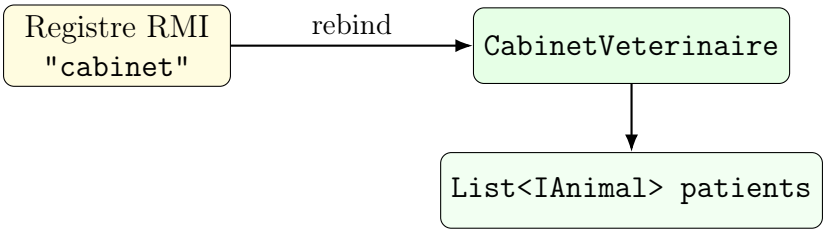


FIGURE 4 – Le cabinet comme unique liaison publiée, et point d'accès à tous les patients.

Pour y voir plus clair, j'ai préféré démarrer un nouveau client dédié à cette étape plutôt que d'alourdir celui d'A1/A2 : il récupère le cabinet, liste les patients, puis effectue une recherche sur un patient connu et une autre sur un nom inexistant (`screenshots/A4.1.png` à `A4.3.png`). On a désormais une structure fonctionnelle — mais encore rigide : tous les patients sont créés en dur dans le constructeur du servant, alors qu'un cabinet reçoit des patients au fil du temps.

2.5 Etape A5 : créer un patient depuis le client

`CabinetVeterinaire` reçoit une nouvelle opération, `ajouterPatient(nom, nomMaitre, race, espece)`, appelable à distance. Le point important, et la vraie subtilité de cette étape, c'est que l'animal doit être instancié **dans la JVM du serveur**, pas dans celle du client : un `new Animal(...)` exécuté côté client créerait un objet exporté depuis le processus client, ce qui n'a aucun sens pour un patient censé appartenir au cabinet. C'est donc le servant du cabinet, recevant l'appel distant, qui construit l'animal lui-même.

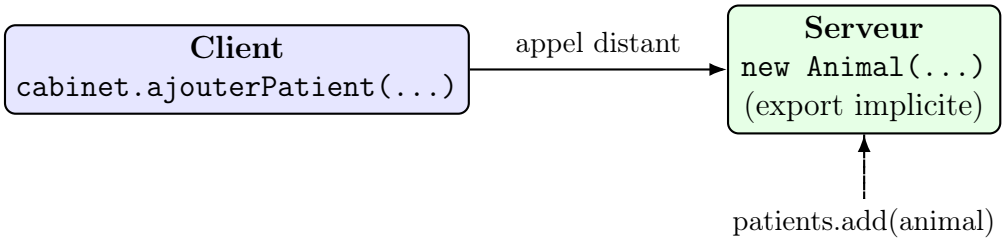


FIGURE 5 – L'animal est construit côté serveur : seul l'appel traverse le réseau.

Côté client, la validation suit la logique du point de contrôle E : on relève le nombre de patients avant l'appel, on ajoute un nouveau patient, on vérifie que le compte a bien augmenté, puis on retrouve ce même patient par une recherche par nom, le tout via une interaction réellement distante, et non une simple trace laissée côté serveur (`screenshots/A5.1.png` à `A5.3.png`).

On dispose maintenant d'un cabinet capable d'accueillir, rechercher et suivre ses patients. L'étape suivante part d'une observation simple : un cabinet a une capacité limitée, et il serait utile que les vétérinaires soient prévenus quand cette capacité est atteinte.

2.6 Etape A6 : alertes, le serveur rappelle ses clients

On veut que tous les vétérinaires connectés soient prévenus automatiquement lorsque le nombre de patients franchit les seuils 100, 500 ou 1 000, à la hausse comme à la baisse. C'est l'occasion de mettre en œuvre le patron **Observer** vu en CM 01, mais cette fois à travers le réseau (CM 02, section 5) : le serveur doit pouvoir appeler une méthode située *chez le client*, alors que jusqu'ici la communication n'allait que dans l'autre sens.

Le principe : le client implémente et **exporte** lui-même un objet `Observateur` (identifié simplement par un nom de vétérinaire, le nom retenu par le sujet, même s'il désigne ici un vétérinaire plutôt qu'un vrai « observateur » au sens strict), puis en transmet le stub au cabinet via `sAbonner`. Le cabinet retient une liste de ces stubs, et peut désormais les rappeler.

Listing 5 – `common/veterinaire/IObservateur.java` et son implémentation côté client

```
public interface IObservateur extends Remote {
    void alerte(int nombrePatients, int seuil, boolean hausse) throws RemoteException
        ;
}

// Cote client : exporte des sa creation (extends UnicastRemoteObject)
public void alerte(int nombrePatients, int seuil, boolean hausse) {
    String sens = hausse ? "franchi A LA HAUSSE" : "repasse EN DESSOUS";
    System.out.println("[" + nomVeterinaire + "] ALERTE seuil " + seuil + " : " +
        sens);
}
```

Chaque appel à `ajouterPatient` déclenche une vérification des seuils : on compare le nombre de patients avant et après l'ajout, et si l'un des trois seuils vient d'être franchi dans un sens ou dans l'autre, le cabinet parcourt sa liste d'observateurs abonnés et appelle `alerte(...)` sur chacun. Ce qui rend le mécanisme réellement distribué, et pas un simple affichage local, c'est que chaque `obs.alerte(...)` est un **véritable appel RMI** : il traverse le réseau jusqu'au stub exporté par le client, et c'est bien le processus du client, pas celui du serveur, qui exécute `System.out.println`. C'est pour cela que l'affichage arrive sur un thread du runtime RMI côté client, indépendant du thread principal : il peut donc s'intercaler avec n'importe quelle autre sortie en cours.

La robustesse compte autant que le mécanisme lui-même : un client abonné peut très bien se fermer brutalement entre deux alertes. L'appel `obs.alerte(...)` échoue alors avec une `RemoteException`, que le cabinet capture pour retirer silencieusement l'observateur devenu injoignable, sans que cela n'affecte les autres abonnés.

Le test avec deux clients abonnés simultanément est concluant (point de contrôle F, `screenshots/A6.1.png` à `A6.3.png`) : le franchissement du seuil de 100 patients s'affiche bien dans les deux consoles clientes, et la fermeture brutale de l'un n'empêche pas l'autre de continuer à recevoir ses alertes.

2.7 Etape A7 : ce qui traverse vraiment le réseau

Cette étape est une expérience plutôt qu'une nouvelle fonctionnalité : elle sert à vérifier, en le faisant échouer volontairement, ce qu'un flux sérialisé transporte réellement.

Protocole. Dans le projet `client` uniquement, j'ai défini `EspeceChien`, une sous-classe d'`Espece` (avec un champ supplémentaire, une race particulière). Le serveur, lui, n'a jamais compilé cette classe. J'ai ensuite envoyé une instance d'`EspeceChien` au cabinet via l'opération d'ajout de patient de l'étape A5.

Résultat observé. L'appel échoue côté client avec une `ServerException`, qui encapsule en réalité une `UnmarshalException` levée côté serveur, elle-même causée par une `ClassNotFoundException: veterinaire.EspeceChien` (point de contrôle G, capture dans `screenshots/A7.2.png`).

Explication. Un flux sérialisé transporte le **nom complet de la classe** et **l'état de l'objet** (la valeur de ses champs), jamais son **bytecode**. Désérialiser consiste, pour le receveur, à retrouver cette classe dans son propre classpath puis à y réinjecter l'état reçu : si la classe lui est inconnue, il n'a tout simplement aucun moyen de reconstruire l'objet, d'où l'exception. C'est exactement la limite énoncée en CM 02 (section 5) entre « envoyer une photo » et « envoyer le moyen de la développer » : la photo (l'état) voyage, mais pas l'appareil qui sait la développer (le code).

Correction. La façon la plus simple de corriger cela est de garantir que toute classe transmise par valeur est connue des **deux** côtés : il suffit de placer `EspeceChien` dans `common` plutôt que dans `client`, exactement comme `Espece`. Une fois les trois projets recompilés, le même appel aboutit normalement. Une alternative plus lourde — le *chargement dynamique de code* via `java.rmi.server.codebase`, vu en CM 02 section 7 — permettrait au serveur de télécharger la classe à la volée depuis le client ; je l'ai explorée à part dans `codebase-demo/` par curiosité, mais ce n'était pas la correction demandée ici, et elle introduit une lourdeur (configuration du `SecurityManager`, codebase à exposer) disproportionnée pour ce besoin précis.

2.8 Etape A8 : une application utilisable, l'interface en ligne de commande

Jusqu'ici, chaque étape a été validée par des appels écrits en dur dans un `main` : c'était suffisant pour tester une fonctionnalité, mais ce n'est pas une application que quelqu'un d'autre pourrait utiliser sans lire ni recompiler le code. L'étape A8 transforme le client en une véritable **interface en ligne de commande** (CLI), organisée en trois responsabilités bien séparées, comme demandé par le sujet :

Classe	Rôle
`ClientMain`	point d'entrée : lit les arguments (hôte, port, nom du vétérinaire), obtient le stub du cabinet par `lookup`, instancie et démarre les deux classes suivantes
`ClientLogic`	logique client : concentre toutes les opérations sur les interfaces distantes
`ConsoleInterface`	interface console : affiche le menu, lit et valide les saisies, affiche les résultats

La contrainte la plus structurante est que `ConsoleInterface` ne doit contenir **aucun appel à** `java.rmi` : je l'ai vérifié directement avec `grep -rl "import java.rmi" client/veterinaire/`, qui ne doit jamais faire remonter ce fichier. Concrètement, cela veut dire que la console ne manipule jamais un `IAnimal` ou un `ICabinetVeterinaire` (dont les méthodes déclarent `RemoteException`), mais uniquement des objets purement locaux que `ClientLogic` lui fournit, des petites classes comme `InfoAnimal` ou `InfoDossier` que j'ai du réécrire pour plus de simplicité mais qui équivaut aux classes précédentes, et une exception non vérifiée, `ErreurServiceDistant` propre à notre CLI.

Listing 6 – Traduction d'une `RemoteException` en exception locale, dans `ClientLogic.java`

```
public InfoAnimal rechercherParNom(String nom) {
    try {
        IAnimal animal = cabinet.rechercherParNom(nom);
        return animal == null ? null
            : new InfoAnimal(animal.getNom(), animal.getNomMaitre(),
                             animal.getRace(), animal.getEspece());
    } catch (RemoteException e) {
        throw new ErreurServiceDistant("La recherche a echoue pour " + nom + ".", e);
    }
}
```

Chaque méthode de `ClientLogic` suit ce même patron : elle attrape la `RemoteException` issue de l'appel distant et la relève sous forme d'`ErreurServiceDistant`, une `RuntimeException` que la console peut attraper sans jamais avoir besoin de connaître `java.rmi`. C'est aussi `ClientLogic` qui gère l'abonnement aux alertes : elle crée et exporte l'`Observateur` au moment de `sAbonner()`, et le désexporte explicitement (`UnicastRemoteObject.unexportObject`) au moment de se désabonner.

Listing 7 – Menu principal de `ConsoleInterface.java`, qui n'importe jamais `java.rmi`

```
private void traiterChoix(String choix) {
    try {
        switch (choix) {
            case "1": lister(); break;
            case "2": rechercher(); break;
            // ...
            case "9": enCours = false; break;
            default: System.out.println("Choix invalide.");
        }
    } catch (ErreurServiceDistant e) {
        System.out.println("Echec de l'operation : " + e.getMessage());
    } catch (RuntimeException e) {
        System.out.println("Saisie invalide : " + e.getMessage());
    }
}
```

Ce double `catch` répond directement à deux exigences du point de contrôle H : une panne distante (serveur arrêté, patient disparu) affiche un message compréhensible sans jamais arrêter le client, tout comme une saisie invalide (un nombre mal formé, par exemple).

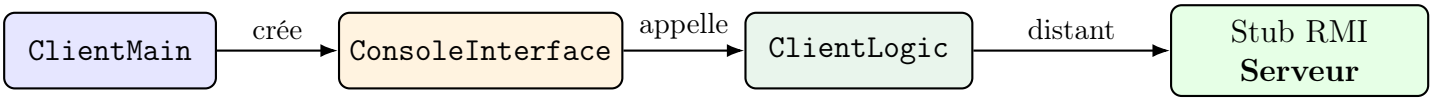


FIGURE 6 – Les trois responsabilités de l'étape A8 : seule `ClientLogic` connaît RMI.

Deux subtilités, soulignées par les indices du sujet, méritent d'être consignées :

— **Une alerte peut arriver pendant une saisie.** L'affichage déclenché par `Observateur.alerte(...)` s'exécute sur un thread du runtime RMI, totalement indépendant du thread qui lit le `Scanner` dans `ConsoleInterface`. Rien n'empêche donc une alerte de s'intercaler au milieu d'un prompt en cours de saisie — c'est visible tel quel à l'exécution, sans code supplémentaire, mais cela vaut la peine d'être noté : une version plus aboutie pourrait synchroniser l'affichage pour ne jamais couper un prompt en cours.

— **Quitter doit vraiment arrêter le processus.** Si le vétérinaire était abonné aux alertes, son `Observateur` est resté exporté : les threads d'écoute non démons ouverts par cet export empêcheraient sinon la JVM du client de se terminer, même après l'option « Quitter » du menu. C'est pourquoi `ConsoleInterface.lancer()` appelle systématiquement `logique.fermer()` en sortie de boucle, qui désabonne puis désexporte l'observateur le cas échéant.

Le point de contrôle H se valide en lançant deux clients CLI dans deux terminaux séparés : on enregistre un patient depuis l'un, on le retrouve et on modifie son dossier depuis l'autre, et les deux consoles reçoivent bien les mêmes alertes de seuil. Arrêter le serveur puis invoquer une opération renvoie un message intelligible plutôt qu'une pile d'exceptions, et choisir « Quitter » termine effectivement le processus client.

Regard critique. La recherche par nom reste ambiguë si deux patients portent le même nom (le premier trouvé l'emporte silencieusement) ; une CLI plus aboutie identifierait chaque animal par un identifiant unique plutôt que par son nom. De même, l'affichage reste minimal (pas de couleurs, pas de tableau formaté) : rien d'ambitieux n'était demandé à ce stade, mais c'est un axe d'amélioration naturel, en particulier en vue de la partie C où une vraie interface graphique vient remplacer cette console.

3 Etape B : assembler l'application avec Spring Boot

Cette partie est bonus, et elle ne remet rien en cause de ce qui précède : le Java RMI de la partie A reste exactement le même, aucune classe de `common`, `server` ou `client` n'a changé de comportement. Ce que Spring Boot modifie, c'est uniquement la manière dont les objets sont construits et assemblés, pas la manière dont ils communiquent à distance. Je m'appuie dans cette partie sur l'annexe 2 du sujet, qui fixe les versions et la structure Maven attendues, et je ne reprends pas ici tout son contenu, seulement les choix que j'ai faits et les difficultés rencontrées.

3.1 Repartir d'un projet Maven propre

Plutôt que de transformer progressivement mes trois dossiers de la partie A, j'ai choisi de reconstruire l'architecture entièrement avec Maven, module par module (`vet-common`, `vet-server`, `vet-client`, avec le `pom.xml` parent et agrégateur à la racine), en suivant la structure de l'annexe. Une fois les quatre `pom.xml` en place, j'ai copié dans chaque module uniquement les classes de la partie A qui avaient encore un rôle clair : les interfaces et types partagés dans `vet-common`, le servant et la configuration dans `vet-server`, la logique client et la console dans `vet-client`.

Ce choix m'a permis de laisser de côté tous les petits clients de test écrits au fil des étapes A1 à A7 (`Client_A1A2A3.java`, `Client_A4A5A7.java`, et les autres variantes du même genre), qui n'avaient plus de raison d'exister une fois l'étape A8 terminée. Le résultat est un projet beaucoup plus lisible, où chaque module ne contient que ce qui est réellement utilisé par l'application finale, sans classes mortes ni doublons de `main`. Cela a aussi forcé une relecture complète de mon code de la partie A : en ne recopiant que ce qui servait encore, j'ai retrouvé deux ou trois incohérences de nommage entre `ClientLogic` et `ConsoleInterface` que je n'avais pas vues en travaillant étape par étape, et que j'ai corrigées à cette occasion.

3.2 Ce que Spring Boot a changé concrètement

Côté serveur, le servant devient un bean `@Service` qui étend toujours `UnicastRemoteObject` et implémente toujours `ICabinetVeterinaire` : Spring se charge de le construire et de lui injecter ses dépendances par constructeur, mais l'export RMI a lieu exactement comme en partie A. Le registre et la publication (`rebind`) sont regroupés dans une classe `@Configuration`, et les paramètres (port, nom du service) viennent désormais d'`application.properties` plutôt que d'être écrits en dur. Côté client, le principe est le même : un `lookup` unique dans une classe de configuration fournit le stub du cabinet, injecté ensuite partout où `ClientLogic` en a besoin, au lieu d'être refait à chaque appel.

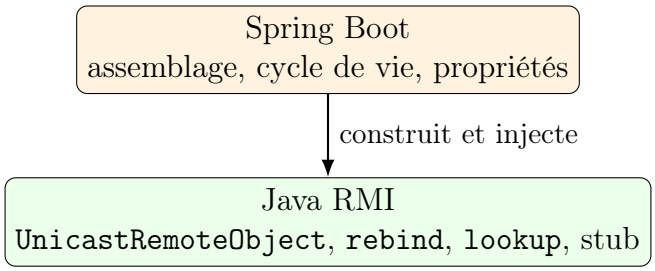


FIGURE 7 – Spring Boot assemble les objets, le Java RMI de la partie A reste inchangé en dessous.

3.3 Un problème rencontré, le client graphique qui refusait de s'ouvrir

En installant la partie B, j'ai suivi l'indication de l'annexe et ajouté `spring.main.headless=false` dans `application.properties` du client, pour pouvoir ensuite lancer une interface graphique Swing (partie C) par dessus ce client Spring Boot. Cela n'a pourtant pas suffi, le client refusait toujours de s'ouvrir avec une `java.awt.HeadlessException`, alors même que la propriété était bien présente dans le fichier.

La cause tient à l'ordre dans lequel les choses s'initialisent, le mode headless de Java est décidé très tôt au démarrage de la JVM, avant même que Spring ait fini de lire ses fichiers de configuration, donc la propriété posée dans `application.properties` arrivait trop tard pour avoir un effet. La solution a été de passer cette même propriété directement en argument JVM au lancement, pour qu'elle soit lue dès le départ, plutôt qu'en argument d'application Spring :

Listing 8 – Lancement du client avec le mode headless désactivé dès la JVM

```
mvn -pl vet-client spring-boot:run \
    -Dspring-boot.run.jvmArguments="-Djava.awt.headless=false"
```

Cette expérience illustre bien la remarque de l'annexe, Spring Boot gère l'assemblage et le cycle de vie de l'application, mais certains réglages bas niveau de la JVM restent en dehors de son contrôle, et doivent être fixés au niveau du lancement du processus plutôt que dans les propriétés de l'application.

3.4 Bilan

Globalement, cette partie B n'a rien changé à la logique distribuée construite en partie A, elle en a simplifié l'assemblage : moins de code d'initialisation écrit à la main (création du registre, publication, lecture d'arguments), une configuration externalisée et donc plus facile à faire varier d'un environnement à l'autre, et un projet Maven reproductible pour quiconque voudrait le relancer avec les mêmes versions. En contrepartie, repartir de zéro avec Maven demande un peu de rigueur au départ (bien distinguer héritage de `parent` et dépendance vers `vet-common`), mais le nettoyage que cela a entraîné sur mes anciens clients de test a clairement rendu le projet plus propre et plus simple à présenter.

4 Etape C : une interface graphique pour le client

Cette dernière partie est également bonus. Elle ne touche ni au serveur ni au contrat RMI, elle habille le client Spring Boot de la partie B d'une interface graphique Swing, en remplacement de la console de l'étape A8.

4.1 Ce qui a été implémenté

Pour commencer, il faut savoir que je n'ai aucune connaissance de Swing, et que cette étape à été une grosse étape d'apprentissage, qui a été énormément accompagné par l'IA pour ce qui est du code. Donc `VeterinaireGUI` est une fenêtre Swing qui reprend exactement les opérations de la console, affichage de la liste des patients, recherche par nom, consultation et modification du dossier de suivi, abonnement et désabonnement aux alertes. Le point de conception central est que cette interface ne connaît toujours pas RMI : elle appelle les mêmes méthodes de `ClientLogic` que `ConsoleInterface` à l'étape A8, et c'est toujours `ClientLogic` qui traduit les `RemoteException` en `ErreurServiceDistant`. Je n'ai donc dupliqué aucune logique métier, seule la façon de présenter les résultats change.

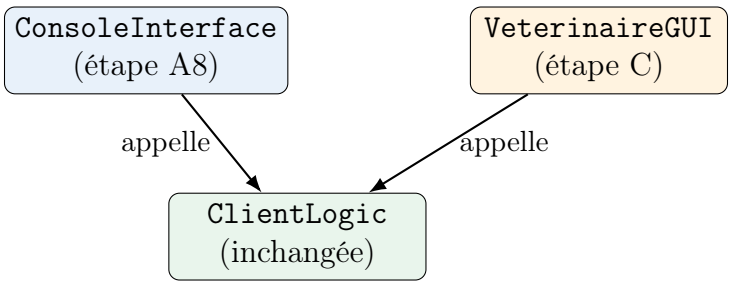


FIGURE 8 – La console et l'interface graphique sont deux présentations interchangeables de la même `ClientLogic`.

Pour ne jamais geler la fenêtre pendant un appel RMI, chaque opération déclenchée depuis l'interface (recherche, ajout, modification du dossier) passe par un `SwingWorker` : le travail réseau s'exécute en arrière plan, et seul le résultat revient sur le thread Swing. Les alertes, elles, arrivent sur un thread du runtime RMI comme expliqué en A6, donc tout affichage déclenché par une alerte passe systématiquement par `SwingUtilities.invokeLater`, pour rester conforme à la règle de Swing selon laquelle seul le thread d'affichage doit toucher les composants graphiques.

Listing 9 – Principe du `SwingWorker` utilisé pour chaque appel RMI dans `VeterinaireGUI.java`

```
private <T> void lancerTache(Tache<T> tache, ActionResultat<T> surSucces) {
    new SwingWorker<T, Void>() {
        protected T doInBackground() { return tache.executer(); }
        protected void done() {
            try { surSucces.accepter(get()); }
            catch (Exception e) { afficherErreur(e); }
        }
    }.execute();
}
```

4.2 Des améliorations ajoutées de ma propre initiative

Au delà de ce qui était strictement demandé par le sujet, j'ai choisi d'ajouter plusieurs comportements à cette interface, pour la rendre réellement utilisable plutôt que simplement fonctionnelle. Je les précise ici explicitement puisqu'ils ne viennent pas d'une consigne mais d'une envie personnelle d'aller un peu plus loin.

- **Validation des champs vides.** Le formulaire d'ajout de patient refuse désormais la saisie si un champ obligatoire est vide, avec un message clair, plutôt que de laisser l'opération échouer silencieusement ou de créer un patient avec des champs incohérents.
- **Retrait d'un patient.** J'ai ajouté une opération `retirerPatient`, de bout en bout (interface, `ClientLogic`, `ICabinetVeterinaire`, `CabinetService`, `PatientRegistry`), avec une confirmation avant suppression, ce qui n'existait pas du tout dans le sujet initial.
- **Messages d'état sur l'abonnement.** S'abonner alors qu'on l'est déjà, ou se désabonner alors qu'on ne l'est pas, affiche désormais un message explicite (« déjà abonné », « déjà désabonné ») au lieu de relancer silencieusement l'opération ou de risquer un doublon dans la liste des observateurs du cabinet.
- **Un bouton de test à quatre vingt dix neuf patients.** Pour vérifier le franchissement des seuils d'alerte sans avoir à cliquer quatre vingt dix neuf fois sur le formulaire, un bouton de test ajoute d'un coup un grand nombre de patients, ce qui a été très utile pour valider le point de contrôle F de la partie A sans perdre de temps à chaque nouvelle vérification.
- **Deux fenêtres clientes indépendantes.** L'interface graphique peut être lancée deux fois, comme deux processus JVM distincts, chacun avec son propre nom de vétérinaire passé en argument, pour simuler deux praticiens connectés en même temps au même cabinet et vérifier que les alertes et les dossiers partagés se comportent bien comme prévu en A3 et A6. C'est un point déja prévu par la consigne en soi mais il fallait s'assurer que graphiquement rien ne soit laissé au hasard, donc je l'ajoute dans cette section.
- **La recherche sélectionne le résultat.** Une recherche fructueuse ne se contente plus d'afficher le patient trouvé dans un panneau séparé, elle sélectionne et fait défiler automatiquement la liste jusqu'à la ligne correspondante, pour que l'utilisateur retrouve visuellement le patient dans son contexte.
- **Actualiser le dossier suivi à chaque ajout** Anodin mais super utile, dès lors qu'on change l'état de santé du patient ou son historique d'observation, au lieu d'à chaque fois écrire, valider puis consulter le dossier pour actualiser, tout se fait d'un coup à la validation. Simple mais efficace.

4.3 Validation

Chaque amélioration a été testée manuellement, formulaire vide refusé avec message, patient retiré puis recherche confirmant sa disparition, double abonnement puis double désabonnement affichant bien

les messages attendus, bouton de test suivi d'une vérification des alertes dans les deux fenêtres clientes ouvertes simultanément, et recherche d'un patient existant suivie d'une vérification visuelle de la sélection dans la liste. J'ai eu aussi le droit à des yeux extérieurs sur la partie interface graphiques pour m'aider à me rendre compte de ce qui n'allait pas sur certains axes ou même ce qui pourrait être utile / agréable.

4.4 Axes d'amélioration envisageables

Cette interface reste un client assez simple, et plusieurs pistes mériteraient d'être creusées pour en faire un outil réellement destiné à un cabinet :

- **Rappels automatiques par email.** Une tâche planifiée côté serveur pourrait parcourir les dossiers de suivi et envoyer un email de rappel de vaccin au maître d'un animal lorsque la date prévue approche, en s'appuyant par exemple sur une bibliothèque comme Jakarta Mail, ce qui demanderait d'ajouter une donnée de date de dernier vaccin dans `IDossierSuivi` et un service d'envoi côté serveur, indépendant du mécanisme RMI.
- **Persistance des données.** Actuellement tout est perdu si le serveur s'arrête, une base de données ou au minimum une sérialisation sur disque du registre de patients permettrait de survivre à un redémarrage.
- **Authentification des vétérinaires.** Rien n'empêche aujourd'hui n'importe quel client de se faire passer pour n'importe quel vétérinaire, un minimum d'identification serait nécessaire dans un contexte réel.
- **Tests automatisés.** L'ensemble du projet a été validé manuellement jusqu'ici, des tests unitaires sur `ClientLogic` et `PatientRegistry`, ainsi que des tests d'intégration lançant un vrai registre RMI en local, sécuriseraient les évolutions futures.
- **Ergonomie plus poussée.** Un thème sombre, des raccourcis clavier, ou un tri des colonnes de la liste de patients amélioreraient le confort d'utilisation sans toucher à l'architecture.

5 Synthèse

Au terme de ce TP, l'ensemble des fonctionnalités demandées par la partie A fonctionne et a été validé par les points de contrôle A à H, objet distant et stub proxy, copie par valeur de l'espèce, dossier de suivi partagé, cabinet comme point d'entrée unique, création de patient côté serveur, alertes par rappel multi clients, limite de la sérialisation sans le bytecode, et interface en ligne de commande sans dépendance à `java.rmi`. Les parties bonus B et C fonctionnent également de bout en bout, projet Maven multi modules assemblé par Spring Boot, et interface graphique Swing réutilisant la même logique client.

La principale difficulté rencontrée a été l'exception `HeadlessException` au lancement du client graphique sous Spring Boot, déjà détaillée en partie B, résolue en comprenant que le mode headless de la JVM se fixe avant la lecture des propriétés Spring, donc en passant l'argument directement à la JVM plutôt qu'au fichier de configuration. Une autre difficulté, plus conceptuelle, a été de choisir si le dossier de suivi devait être un objet distant ou sérialisable, j'ai dû relire plusieurs fois la section correspondante du cours pour être sûr de bien comprendre la conséquence du choix sur le partage des modifications entre clients, avant de me décider pour l'objet distant en A3.

Rien dans le périmètre demandé par la partie A n'est resté non fonctionnel. Les limites qui subsistent sont surtout des limites de confort, recherche par nom ambiguë en cas de doublon, pas de persistance, pas de tests automatisés, plutôt que des manques fonctionnels par rapport au sujet.

Avec le recul, plusieurs points mériteraient d'être améliorés si je devais reprendre ce projet. Sur l'architecture, le choix de tout faire reposer sur le nom du patient comme identifiant fonctionne pour un TP mais ne résiste pas à un vrai usage, un identifiant unique généré à la création serait plus robuste. Sur la robustesse, le serveur ne supporte pas d'accès concurrents vraiment intensifs sur la liste de patients, une structure thread safe plus poussée qu'une simple liste synchronisée serait préférable si le nombre

de clients devait beaucoup augmenter. Sur la maintenabilité, le dossier de suivi reste assez pauvre, un historique de dates et d'auteurs des observations plutôt que de simples chaînes de caractères rendrait le suivi plus exploitable. Sur l'ergonomie, déjà évoquée en partie C, l'interface reste fonctionnelle mais minimale. Enfin sur les tests, l'absence totale de tests automatisés est la limite la plus nette de ce projet, chaque étape ayant été validée manuellement au fil de l'avancement plutôt que par une suite de tests rejouable.

En somme, j'ai pu répondre à la problématique principale qui était la conception d'une application en JAVA RMI natif et toutes ses décisions de conception. J'ai pu comprendre l'utilité de creuser sur comment on pose les fondations d'une application concrète et comment, étape par étape, on avance sur le développement, de plusen plus dans le détail. J'ai appris qu'aucun point ne doit être pris à la légère, et que la différence se joue dans les détails que l'on souhaite mettre en place.

6 Usage des outils d'intelligence artificielle générative

J'ai utilisé Claude (Anthropic) au cours de la rédaction de ce compte rendu, uniquement pour m'aider sur la mise en page et la correction du document LaTeX, relecture orthographique et grammaticale, mise en forme des sections, ajout des schémas illustrant mon architecture et mise en forme des extraits de code déjà écrits par mes soins. L'intégralité des idées présentées dans ce rapport, des décisions de conception, ainsi que toute la logique du code du projet (Java RMI, Spring Boot, Swing) sont entièrement de moi, Claude a participé à l'écriture du code source du projet uniquement côté client ou interface GUI, mais pas aux choix d'architecture / logique. Côté README.md, je lui ai fourni mon architecture, mes commandes pour lancer le serveur et les clients et il a essayé de remttre cela dans les normes du README. Il m'a également aidé à comprendre Swing et les pom.xml de l'annexe, assez rigides à comprendre. Si d'autres aspects semblent tendre vers de l'IA, je garantis qu'ils sont fortuites.

7 Références

— Sujet du TP1, *HAI704I, Architectures logicielles distribuées*, Bachar Rima, Université de Montpellier, année 2026-2027.
— Annexe 2, *Java RMI avec Spring Boot*, même unité d'enseignement, année 2026-2027.
— Support de cours CM02, *Java RMI*, même unité d'enseignement, année 2026-2027.
— Documentation officielle Oracle, *Java Remote Method Invocation (RMI)*, docs.oracle.com.
— Documentation officielle Spring Boot, spring.io/projects/spring-boot.

Table des figures

1 Séparation entre les contrats partagés, le client et le serveur. 3
2 Architecture RMI de base pour l'objet distant `Animal` (point de contrôle A, voir `screenshots/A1.1.png`). 4
3 Différence d'identité d'objet (`identityHashCode`) suite à la sérialisation. 5
4 Le cabinet comme unique liaison publiée, et point d'accès à tous les patients. 6
5 L'animal est construit côté serveur : seul l'appel traverse le réseau. 7
6 Les trois responsabilités de l'étape A8 : seule `ClientLogic` connaît RMI. 10
7 Spring Boot assemble les objets, le Java RMI de la partie A reste inchangé en dessous. . . 11
8 La console et l'interface graphique sont deux présentations interchangeables de la même `ClientLogic`. 12